System Verification and Validation Plan for Software Engineering

Team #16, The Chill Guys
Hamzah Rawasia
Sarim Zia
Aidan Froggatt
Swesan Pathmanathan
Burhanuddin Kharodawala

March 10, 2026

Revision History

Date	Developer	Notes
October 24th	Swesan Pathmanathan, Aidan Froggatt	Completed System Tests section
October 25th	Hamzah Rawasia	Completed Plan sections 2.2, 2.3, 2.4, 2.6
October 26th	Sarim Zia	Added Plan sections 2.1, 2.5, 2.7
October 26th	Burhan Kharodawala	Added Sections 1 and 5
October 27th	All	Added Reflections
March 8th	Burhan Kharodawala	Incorporated Feedback For Requirements

Contents

1 General Information

1.1 Summary

The Verification and Validation plan sets a strong basis for documenting the testing strategies to test key features of this application. It aims to discuss an approach to test the correctness, validity, and reliability of the application and its features. Hitchly is a rideshare application which aims to help McMaster students, staff and faculty members to get an easy, convenient and trustworthy platform to carpool to and from McMaster University. Some of the key functions of this application are to match riders using the user's timetable, location, and user preferences. Another important feature that makes the application more trustworthy is the McMaster official email verification system. This ensures that each user accessing this application is truly associated with the university. The application also aims to provide a cost calculator for drivers to estimate their total cost of carpooling which they can use to set their rideshare prices. Lastly, it includes the payment integration feature which allows for a smooth transaction between drivers and riders.

1.2 Objectives

The Verification and Validation plan's main objectives are as follows:

- **Software Correctness:** Build confidence in the software's correctness by verifying and validating its critical requirements, ensuring that the application does what it claims to do.
- **Features Validation:** Validate the implemented features to ensure they provide a smooth and comfortable user experience.
- **Confidence in Stakeholders:** Build confidence among the stakeholders by providing proof of validation for all software requirements for the project.
- **Validate Components:** This plan aims to verify and confirm the proper implementation and integration of critical components like the matching algorithm, cost calculator, and payment interface.
- **Large Scale Concurrent Testing:**

- **Comprehensive Data Testing:**

The out of scope objectives are as follows:

- **Large Scale Concurrent Testing:** Due to the limited time and resource, performing large scale testing with users accessing the system concurrently to ensure server capabilities may be valuable but avoided.
- **Comprehensive Data Testing:** The validation of secured data encryption may just be limited to functional testing due to the limited resources and time for this project. It will just be limited to the verification of user authentication.

1.3 Challenge Level and Extras

There will be two extras for this project. Both documents play an important role in making the application as user friendly as possible.

- **User Manual:** This will help users navigate the application and provide a brief description of the key features to allow for a smooth and comfortable user experience.
- **Usability Testing Report:** This will ensure that users have a positive and smooth interaction with the application by observing the users while using the application. Moreover, opinions and feedback from the user will be considered and documented to improve the usability of this application. This will help evaluate the user-friendliness of the key features of the application.

1.4 Relevant Documentation

- **Problem Statement and Goals:** This document is very important in providing a layout for documenting the main problem that the application aims to solve and the main goals it plans to achieve. This provides a starting point for testing and highlights the desired results through the identification of goals in the document Kharodawala et al. (2025e).
- **Development Plan:** This document provides a timeline for the development of the project. It ensures that the project is on track with its

development and testing. It also provides a timeline for the tests based on the timeline for the developement of the application Kharodawala et al. (2025a).

- **Software Requirements Specification:** This document aims to breakdown the application's key features and functionality into two-part, functional and non-functional requirements. This plays a key role in listing all key functionalities and planning requirements-based testing Kharodawala et al. (2025f).

- **Hazard Analysis:** This document identifies the key hazards involved with the development of this application. It helps provide a list of vulnerabilities that must be addressed and ruled out through testing Kharodawala et al. (2025b).

- **Module Guide:** This document provides a description of the division of the application into modules. This is important as it provides a clear basis for planing and running module-based and component-based testing Kharodawala et al. (2025c).

- **Module Interface Specification:** This document dives deep into exploring the interface specifications between modules. This would be important to plan testing functionality and interactions between two modules Kharodawala et al. (2025d).

- **Usability Testing Report:** This documents the feedback and observation from usability testing. This is an additional test which ensures the user compatibility of the application Kharodawala et al. (2025g).

2 Plan

This section discusses the team's plan for verification and validation of process implementation. It covers the teams and their roles, the SRS verification plan, the design verification plan, the verification and validation verification plan, the implementation verification plan, automated testing and verification tools, and the software validation plan.

2.1 Verification and Validation Team

Dr. Spencer Smith:

- Advisor and primary documentation reviewer. Provides guidance for project direction, documentation structure, and technical feedback throughout the software development lifecycle.

Lucas Dutton:

- Advisor and primary documentation reviewer. Provides guidance for project direction, documentation structure, and technical feedback throughout the software development lifecycle.

Sarim Zia:

- Responsible for reviewing team members' pull requests to suggest improvements and/or fixes to code and documentation. Ensure smooth integration of reviewed features. Lead internal beta testing throughout the development phase and lead public beta testing to collect and review all relevant feedback.

Aidan Froggatt:

- Responsible for reviewing team members' pull requests to suggest improvements and/or fixes to code and documentation. Oversee smooth integration of reviewed features through creating rulesets. Conduct internal beta testing throughout the development phase.

Swesan Pathmanathan:

- Responsible for reviewing team members' pull requests to suggest improvements and/or fixes to code and documentation. Ensure smooth integration of reviewed features. Conduct internal beta testing throughout the development phase and focus emphasis on user interface responsiveness.

Hamzah Rawasia:

- Responsible for reviewing team members' pull requests to suggest improvements and/or fixes to code and documentation. Ensure smooth integration of reviewed features. Conduct internal beta testing throughout the development phase and focus emphasis on functional feature validation.

Burhanuddin Kharodawala:

- Responsible for reviewing team members' pull requests to suggest improvements and/or fixes to code and documentation. Ensure smooth integration of reviewed features. Conduct internal beta testing throughout the development phase and focus emphasis on non-functional feature validation.

2.2 SRS Verification

We want to ensure that the SRS document is complete, accurate, and consistent, which will be verified using the following approaches:

Peer Review: The team will rely on feedback from classmates that are from other capstone teams for peer evaluation. It is expected that they will provide feedback on the clarity and completeness of each section, going through and finding any inconsistencies and flaws. Based on their findings, GitHub issues will be opened to provide specific suggestions for improvement and any aspects that can be improved.

Stakeholder Feedback: The primary stakeholder(s) of our project are McMaster University students and faculty that commute to campus. As such, we will share the SRS document with a sample of chosen stakeholders. This is important to ensure that the SRS reflects actual stakeholder needs and expectations rather than internal assumptions. The selected users will be able to give specific feedback on which features, requirements, and workflows meet their expectations of the functionality of the system. Any conflicts that may arise between user expectations and written requirements will trigger a revision of the corresponding sections. As for testing the requirements, users will be given an opportunity to use the system and perform basic testing of functionality with set cases given to each user, where they will be able to see the workflow of the application first-hand and provide feedback on the functionality and whether it meets the requirements stated in the SRS.

Team Validation: The team will also meet internally to discuss the SRS and ensure the content is consistent with the intended functionlity and capabilities of the application, and is consistent throughout. Additionally, the team will also conduct our own testing using the system, especially for non-

functional requirements to ensure that the application meets the performance requirements, as well as the overall functionality of each major component of the system.

SRS Verification Checklist

- [ ] Are all of the core features (Matchmaking, scheduling, safety, payments, etc.) clearly described?
- [ ] Is every requirement unique and traceable?
- [ ] Has feedback from the classmate peer review been addressed?
- [ ] Are the requirements clearly defined and measurable?
- [ ] Are the inputs and outputs for important use cases clearly defined?
- [ ] Are there any conflicting requirements and/or assumptions?

2.3 Design Verification

To ensure the design is functional and up to standards, the team will conduct several reviews with all developers to thoroughly go over the technical soundness of the design and major artifacts, such as the system's architecture, database schema, API specifications, and the UI/UX design. For the UI/UX design, it is especially important that we receive feedback from stakeholders to validate the user experience and flow. We will provide users with key user stories and the corresponding interface which is used to complete them so that they are able to get a feel of the functionality of the design. It is important that the team documents all feedback and any confusion from the users to validate the usability of the design. There will also be feedback received from peers on other capstone teams to verify design components, especially written components and diagrams.

Design Verification Checklist:

- [ ] Does the database schema correctly model necessary fields for required data, and appropriate relationships?
- [ ] Does the API design include secure, authenticated endpoints for core components and the functionality?

[ ] Does the system architecture clearly define how components will interact and how key features such as the matchmaking engine will work?

[ ] Do the UI wireframes clearly and intuitively display all core features and workflows?

2.4 Verification and Validation Plan Verification

To ensure that the Verification and Validation plan is complete, correct, and effective, the team will hold an internal review meeting to ensure that the document is complete and feasible. We will also get peer feedback from classmates where they will identify areas of the plan which are unclear, incomplete, or needs improvement. To keep track of the feedback from peers, GitHub issues will be used so that peers can clearly communicate their thoughts. Additionally, the team will ensure that the tests have complete coverage of requirements, and will also make use of mutation testing to verify the quality and effectiveness of the unit tests in the plan. This involves introducing small faults into the code, such as changes in the matchmaking algorithm to modify efficiency and/or effectiveness and running the test suite against it to validate that it is robust.

Verification and Validation Plan Verification:

[ ] Do the requirements in the SRS have at least one corresponding test case?

[ ] Are all test cases clear, specific, and testable, with defined expected outcomes?

[ ] Is the plan for mutation testing feasible?

[ ] Has feedback from the peer review been addressed and closed?

2.5 Implementation Verification

- **Code Walkthrough/Inspection:** New Code PR's will be reviewed by at least two team members before the change is merged into the main branch. The code will be reviewed via pull requests on GitHub and will include thoughtful titles and comments to ensure the reviewers are

able to understand the changes. If any further clarification is required, the developer will conduct a code walkthrough. All feedback will be conveyed via GitHub, ensuring a standardized review procedure. Only approved code will be merged into the main branch.

- **Unit Testing:** Unit tests will be used to verify the expected behavior of the application. Each module will have a corresponding unit test suite. Unit test plans are outlined in Section 5 of this document.
- **System Testing:** System testing will ensure all functional and non-functional requirements are met by the application. Functional testing will verify key workflows, and non-functional testing will include performance, usability, and security checks. System test plans are outlined in Section 4 of this document.
- **Static Analyzers:** Static analysis tools will be used to analyze the code for formatting and good coding practices. Reports generated from static analysis will be reviewed periodically to ensure high code quality. Specific tools are listed under Section 3.6.

2.6 Automated Testing and Verification Tools

To ensure high quality, correct, and maintainable codebase, the team will use the following tools for automated testing and code verification, which were also briefly outlined in the Development Plan document.

Unit Testing Framework: Jest will be the primary framework for running unit tests for both the React Native frontend and Node.js backend logic. We will write unit tests for important business logic such as the cost-sharing calculation, parsing user timetables, and individual React components.

Backend Integration Testing: Supertest will be used to conduct integration testing on our backend API by simulating HTTP requests and verifying the responses. We will create test suites to verify that all API endpoints are functioning as expected, which includes testing for correct data responses and error handling.

Static Analysis and Formatting: ESLint and Prettier will ensure the codebase remains consistent, readable, and without common errors. ESLint

will be configured to analyze our code for potential bugs and help enforce best practices, and Prettier will format code to maintain a consistent style across the codebase.

Jest will also be used for code coverage by utilizing the integrated coverage reporter to measure the percentage of our codebase that is executed by our unit and integration tests. Finally, we will also use Github Actions for Continuous Integration to automate the testing and verification process.

2.7 Software Validation

Our validation plan for the software includes extensive user testing and review sessions with all relevant stakeholders to ensure that all functional and non-functional requirements are met.

- **User Testing:** Hitchly relies on extensive external user testing to validate that the system functions correctly under real-world conditions. Beta testing will begin early with a focus on core functionality. Relevant feedback from beta users will be incorporated throughout the development phase. Internal beta testing will also be performed to validate feature functionality and consistency before releasing it for public testing.
- **Stakeholder Review Sessions:** Our stakeholder review sessions will be conducted regularly with our assigned TA to evaluate the system's progress against expected targets. These meetings will be used to refine the system's design and ensure stakeholder expectations are met.

3 System Tests

Purpose: Verify that Hitchly meets its defined functional and non-functional requirements.

Structure:

- Section 3.1 tests core functional requirements (verification and matching).
- Section 3.2 tests key non-functional qualities (reliability and usability).

- Section 3.3 maps each test to its source requirement for traceability.

Reference: Section 1.4 (G.4) *Functionality Overview* defines the functional and non-functional requirements.

3.1 Tests for Functional Requirements

Objective: Confirm correct operation of primary system functions—user verification and ride matching.

Justification: These address Hitchly's two most critical risks trust and core value creation.

Sub-areas:

- 3.1.1 User Verification Tests
- 3.1.2 Ride Matching Tests

3.1.1 Area of Testing 1 – User Verification

Purpose: Ensure only McMaster-affiliated users and verified drivers access the system. Covers: FR-1 (User verification) from Section 1.4 (G.4).

1. **Test Case 1 – Valid Sign-Up**
   Control: Automatic
   Initial State: No existing account.
   Input: Valid McMaster email + password + driver license (upload for drivers).
   Expected Output: Verification email sent → account activated after confirmation.
   Test Case Derivation: Based on credential and email-domain constraints defined in Section 1.4 (G.4).
   How test will be performed: Simulate sign-up, check database entry and email receipt.

2. **Test Case 2 – Invalid Verification**
   Control: Automatic
   Initial State: No existing account.
   Input: Non-McMaster email or invalid license format.

Expected Output: Account creation rejected with error message.
Test Case Derivation: Derived from input constraints and driver-license validation rules in Section 1.4 (G.4).
How test will be performed: Input invalid data, verify system blocks progress.

3.1.2 Area of Testing 2 – Ride Matching

Purpose: Confirm that the matching algorithm correctly proposes compatible rider–driver pairs. Covers: FR-3 (Ride matching) and FR-4 (Cost estimate) from Section 1.4 (G.4).

1. **Test Case 1 – Successful Match**
   Control: Automatic
   Initial State: Verified users with valid profiles and schedules.
   Input: Rider and driver with overlapping routes/times.
   Expected Output: Match proposal generated with computed cost share.
   Test Case Derivation: Based on matching criteria and cost-sharing logic specified in Section 1.4 (G.4).
   How test will be performed: Run matching routine, check match records and cost calculation.

2. **Test Case 2 – No Compatible Match**
   Control: Automatic
   Input: Rider and driver with non-overlapping schedules.
   Expected Output: "No match found" message.
   Test Case Derivation: From matching constraints outlined in Section 1.4 (G.4).
   How test will be performed: Submit non-overlapping profiles and observe system response.

3. **Test Case 3 – User Rejection**
   Control: Automatic
   Input: Rider and driver sent a rideshare request.
   Expected Output: "User rejected" message.
   Test Case Derivation: From matching constraints outlined in Section

1.4 (G.4).
How test will be performed: Submit a rideshare request with a user rejection and observe system response.

3.1.3 Area of Testing 3 - Profile Input

Purpose: Confirm that the user is able to input and update their profile information correctly. Covers: FR-2 (Profile and Scheduling Input).

1. **Test Case 1 - Driver Verification**
   Control: Automatic
   Initial State: Driver profile with valid license.
   Input: Driver profile with valid license.
   Expected Output: Driver verified and added to system.
   Test Case Derivation: Based on verification criteria specified in Section 1.4 (G.4).
   How test will be performed: Submit driver profile and verify system response.

2. **Test Case 2 - Profile Update**
   Control: Automatic
   Initial State: Existing user with valid profile.
   Input: Updated profile information (e.g., contact details, driver/rider preference).
   Expected Output: Profile updated in system.
   Test Case Derivation: Based on update mechanisms specified in Section 1.4 (G.4).
   How test will be performed: Modify profile fields and verify changes are saved.

3.1.4 Area of Testing 4 - Cost Estimation

Purpose: Confirm that the user is able to get a cost estimation of a given ride. Covers: FR-4 (Cost estimate) from Section 1.4 (G.4).

1. **Test Case 1 - Cost Calculation**
   Control: Automatic
   Initial State: Valid ride details (distance, duration, time of day).
   Input: User submits a ride request.
   Expected Output: Accurate cost estimate based on the provided details.
   Test Case Derivation: The final cost is estimated using the logic described in Section 1.4 (G.4).
   How test will be performed: Verify calculated cost once the ride is selected and accepted.

2. **Test Case 2 - Dynamic Cost Calculations**
   Control: Automatic
   Initial State: Existing user with valid profile.
   Input: New rider is added for the same ride.
   Expected Output: Cost estimate updated in system.
   Test Case Derivation: Based on cost estimation logic specified in Section 1.4 (G.4).
   How test will be performed: Add a new rider to an existing ride and verify the updated cost estimate.

3.1.5 Area of Testing 5 - Rating & Reporting

Purpose: Confirm that a user is able to rate, review and report another user for a given ride. Covers: FR-5 (Review and Ratings) from Section 1.4 (G.4).

1. **Test Case 1 - Reviews and Rating**
   Control: Automatic
   Initial State: User has completed a ride.
   Input: User submits a review and rating.
   Expected Output: Review and rating submitted successfully.
   Test Case Derivation: Based on the review and rating criteria in Section 1.4 (G.4).
   How test will be performed: Submit a review and rate the user and observe the state.

2. **Test Case 2 - Reporting**
   Control: Automatic
   Initial State: User has completed/accepted a ride
   Input: Report submitted successfully.
   Expected Output: "Report Submitted" message.
   Test Case Derivation: Based on the review and rating criteria in Section 1.4 (G.4).
   How test will be performed: Submit a report for a user and wait for the submission confirmation alert

3. **Test Case 3 - Report in Admin Dashboard**
   Control: Automatic
   Initial State: User has completed/accepted a ride
   Input: Report submitted successfully.
   Expected Output: Display report details in admin dashboard
   Test Case Derivation: Based on the review and rating criteria in Section 1.4 (G.4).
   How test will be performed: Submit a report for a user and observe the admin dashboard

3.2 Tests for Nonfunctional Requirements

Objective: Evaluate system quality beyond correctness—reliability and usability. Approach: Use performance measurement and user feedback methods rather than simple pass/fail criteria.

Sub-areas:

- 3.2.1 Reliability (Matching Correctness)
- 3.2.2 Usability (Low-Friction Flows)

3.2.1 Area of Testing 1 – Reliability (Matching Correctness)

Purpose: Ensure consistent and repeatable ride matching results. Covers: NFR-1 (Reliability) from Section 1.4 (G.4).

1. **Test Case 1 – Repeatability Check**
   Type: Automatic / Dynamic

Initial State: Stable dataset for verified users.
Input: Run matching inputs multiple times.
Expected Result: Identical pairings each run.
How test will be performed: Automated script logs output and compares hash results.

2. **Test Case 2 – Stress Test**
   Type: Dynamic Performance Test
   Input: Simulated peak load ($\geq$ 1000 simultaneous match requests).
   Expected Result: $\geq$ 95% successful matches within response-time threshold.
   How test will be performed: Load-testing tool records failures and latency for reliability assessment.

3.2.2 Area of Testing 2 – Usability (Low-Friction Flows)

Purpose: Validate ease of use for key flows (sign-up, ride offer/request, confirmation). Covers: NFR-2 (Usability) from Section 1.4 (G.4).

1. **Test Case 1 – First-Time User Scenario**
   Type: Manual / Survey-Based
   Initial State: New participants with no training.
   Input: Observe users completing sign-up and ride requests.
   Expected Result: $\geq$ 90% complete tasks without help in $\leq$ 3 minutes.
   How test will be performed: Usability study plus survey metrics (Appendix).

2. **Test Case 2 – Interface Clarity**
   Type: Static Inspection / Heuristic Evaluation
   Input: UI screens and labels.
   Expected Result: No ambiguous labels or layout clutter violations.
   How test will be performed: Walkthrough by UX reviewers using standard heuristics.

3.3 Traceability Between Test Cases and Requirements

Requirement ID	Requirement Description (from 1.4 G.4)	Test Case ID(s)
FR-1	User Verification	3.1.1-1, 3.1.1-2
FR-3	Ride Matching	3.1.2-1, 3.1.2-2
FR-4	Cost Estimate	3.1.2-1
NFR-1	Reliability (Matching Correctness)	3.2.1-1, 3.2.1-2
NFR-2	Usability (Low-Friction Flows)	3.2.2-1, 3.2.2-2

Table 1: Traceability between test cases and requirements as defined in Section 1.4 (G.4) Functionality Overview.

4 Unit Test Description

Unit testing focuses on deterministic modules identified in the MIS, particularly the backend business logic, data validation utilities, service layers, and shared packages. Tests are implemented with Vitest (Jest-compatible) in the respective `tests` and `__tests__` directories for both the backend TRPC routers and shared packages. Dependencies such as the database, Stripe SDK, and Google Maps API are mocked to isolate unit behavior. Tests are organized below by the MIS module they exercise.

4.1 Unit Testing Scope

In scope: All deterministic backend logic in M1 through M12 (excluding M5: Scheduling, which is deferred, and M10: Admin & Moderation, which is covered by system-level tests).
Out of scope: End-to-end UI flows, full Expo mobile rendering, and live Stripe webhook processing (covered by integration and system tests in Section 3).

4.2 Tests for Functional Requirements

4.2.1 Authentication & Verification Module

Test files: `packages/db/tests/validators.test.ts`, `packages/emails/client.test.ts`

1. **test-ut-auth-1**
   Type: Functional, Dynamic, Automatic
   Initial State: System ready for new user registration.
   Input: Valid McMaster email address (`user@mcmaster.ca`) and password.
   Output: Registration validation passes (`success: true`).
   Test Case Derivation: Ensures the system accepts valid university domains as per FR-1.
   How test will be performed: Vitest in `packages/db/tests/validators.test.ts`.

2. **test-ut-auth-2**
   Type: Functional, Dynamic, Automatic
   Initial State: System ready for new user registration.
   Input: Invalid email domain (`user@gmail.com`).
   Output: Registration rejected with a validation error constraint.
   Test Case Derivation: Verifies the system rejects non-McMaster emails to maintain exclusivity.
   How test will be performed: Vitest in `packages/db/tests/validators.test.ts`.

3. **test-ut-auth-3**
   Type: Functional, Dynamic, Automatic
   Initial State: Mocked nodemailer transport.
   Input: OTP code and recipient email address.
   Output: Email sent with correct recipient, OTP embedded in HTML body, and proper HTML document structure.
   Test Case Derivation: Validates that OTP verification emails are correctly formatted and dispatched.
   How test will be performed: Vitest in `packages/emails/client.test.ts`.

4.2.2 User Profile Module

Test files: `apps/api/trpc/routers/__tests__/profile.test.ts`, `packages/utils/index.test.`

1. **test-ut-profile-1**
   Type: Functional, Dynamic, Automatic
   Initial State: Mocked database context with existing user.

Input: Profile metadata updates (bio, faculty, year, role).
Output: Upserts profile data successfully to the database.
Test Case Derivation: Confirms users can correctly initialize and update their core profile data.
How test will be performed: Vitest in `apps/api/trpc/routers/__tests__/profile.test.ts`

2. **test-ut-profile-2**
   Type: Functional, Dynamic, Automatic
   Initial State: Mocked database context.
   Input: Vehicle metadata (make, model, color, plate, seats).
   Output: Upserts vehicle data successfully, correctly rejecting invalid license plates.
   Test Case Derivation: Ensures drivers can correctly add and validate their vehicle details.
   How test will be performed: Vitest in `apps/api/trpc/routers/__tests__/profile.test.ts`

3. **test-ut-profile-3**
   Type: Functional, Dynamic, Automatic
   Initial State: Utility function loaded.
   Input: Standard address components (street, number, city, region, country code).
   Output: Correctly formatted title, subtitle, and full address strings.
   Test Case Derivation: Ensures the address formatting utility produces consistent display strings for user profiles.
   How test will be performed: Vitest in `packages/utils/index.test.ts`.

4. **test-ut-profile-4**
   Type: Functional, Dynamic, Automatic
   Initial State: Utility function loaded.
   Input: Empty or missing address components.
   Output: Returns `null`.
   Test Case Derivation: Validates graceful handling of incomplete location data in user profiles.
   How test will be performed: Vitest in `packages/utils/index.test.ts`.

5. **test-ut-profile-5**
   Type: Functional, Dynamic, Automatic
   Initial State: Mocked database with completed trips and accepted trip requests for the driver.
   Input: Authenticated driver queries their earnings.
   Output: Correct lifetime, weekly, and monthly earnings totals returned based on completed passenger counts; zero returned when no completed trips exist.
   Test Case Derivation: Validates the driver earnings aggregation logic across multiple time windows.
   How test will be performed: Vitest in `apps/api/trpc/routers/__tests__/profile.test.ts`

6. **test-ut-profile-6**
   Type: Functional, Dynamic, Automatic
   Initial State: Mocked database with no completed trips for the driver.
   Input: Authenticated driver queries their earnings.
   Output: All earnings fields return zero; `avgPerTripCents` is zero.
   Test Case Derivation: Validates the zero-state edge case for new drivers with no trip history.
   How test will be performed: Vitest in `apps/api/trpc/routers/__tests__/profile.test.ts`

7. **test-ut-profile-7**
   Type: Functional, Dynamic, Automatic
   Initial State: Authenticated user with mocked database.
   Input: Valid Expo push notification token string.
   Output: Push token updated in the `users` table for the authenticated user.
   Test Case Derivation: Validates the push token persistence flow for enabling notifications.
   How test will be performed: Vitest in `apps/api/trpc/routers/__tests__/profile.test.ts`

8. **test-ut-profile-8**
   Type: Functional, Dynamic, Automatic
   Initial State: Authenticated user.
   Input: Query for ban status.
   Output: Returns {`isBanned: false, reason: null`} (placeholder

implementation).
Test Case Derivation: Validates the stub ban status endpoint returns the expected default.
How test will be performed: Vitest in `apps/api/trpc/routers/__tests__/profile.test.ts`

9. **test-ut-profile-9**
   Type: Functional, Dynamic, Automatic
   Initial State: Authenticated user with mocked database.
   Input: Role switch to "rider".
   Output: Profile upserted with `appRole` set to "rider"; returns {`success: true`}.
   Test Case Derivation: Validates the role switching mutation for the rider role.
   How test will be performed: Vitest in `apps/api/trpc/routers/__tests__/profile.test.ts`

10. **test-ut-profile-10**
    Type: Functional, Dynamic, Automatic
    Initial State: Authenticated user with mocked database.
    Input: Role switch to "driver".
    Output: Profile upserted with `appRole` set to "driver"; returns {`success: true`}.
    Test Case Derivation: Validates the role switching mutation for the driver role.
    How test will be performed: Vitest in `apps/api/trpc/routers/__tests__/profile.test.ts`

4.2.3 Route & Trip Module

Test files: `apps/api/trpc/routers/__tests__/trip.test.ts`, `apps/api/trpc/routers/__tests__` `apps/api/services/__tests__/googlemaps.test.ts`

1. **test-ut-trip-1**
   Type: Functional, Dynamic, Automatic
   Initial State: Mocked database context with verified user.
   Input: Valid trip creation parameters (origin, destination, departure time, max seats).
   Output: Trip created successfully; unverified users, invalid seat counts,

and departure times too soon are rejected.
Test Case Derivation: Verifies trip creation validation and authorization (FR-5).
How test will be performed: Vitest in `apps/api/trpc/routers/__tests__/trip.test.ts`.

2. **test-ut-trip-2**
   Type: Functional, Dynamic, Automatic
   Initial State: Mocked database with multiple trips and requests.
   Input: Query for trips with optional userId filter.
   Output: Correctly returns trips with associated driver info and requests; filtering by userId works.
   Test Case Derivation: Validates trip listing and retrieval logic.
   How test will be performed: Vitest in `apps/api/trpc/routers/__tests__/trip.test.ts`.

3. **test-ut-trip-3**
   Type: Functional, Dynamic, Automatic
   Initial State: Mocked database with existing trip.
   Input: Update request from trip owner vs. non-owner vs. completed trip.
   Output: Successful update for owner of pending trip; rejection for non-owner or non-pending status.
   Test Case Derivation: Ensures proper authorization and state guards on trip mutations.
   How test will be performed: Vitest in `apps/api/trpc/routers/__tests__/trip.test.ts`.

4. **test-ut-trip-4**
   Type: Functional, Dynamic, Automatic
   Initial State: Active trip with accepted riders.
   Input: Cancel request from driver.
   Output: Trip status set to "cancelled"; all pending requests rejected; accepted riders notified.
   Test Case Derivation: Validates the cancellation flow including notification side effects.
   How test will be performed: Vitest in `apps/api/trpc/routers/__tests__/trip.test.ts`.

5. **test-ut-trip-5**
   Type: Functional, Dynamic, Automatic
   Initial State: Trip in "active" status.
   Input: Start trip request from driver.
   Output: Trip transitions to "in_progress"; non-owners and non-active trips are rejected.
   Test Case Derivation: Verifies the trip lifecycle state machine transitions.
   How test will be performed: Vitest in `apps/api/trpc/routers/__tests__/trip.test.ts`.

6. **test-ut-trip-6**
   Type: Functional, Dynamic, Automatic
   Initial State: Trip in "in_progress" with accepted passenger.
   Input: Driver picks up and drops off a passenger.
   Output: Passenger status transitions through "on_trip" to "completed"; pickup without rider confirmation is rejected.
   Test Case Derivation: Validates passenger status transitions and the rider confirmation guard.
   How test will be performed: Vitest in `apps/api/trpc/routers/__tests__/trip.test.ts`.

7. **test-ut-trip-7**
   Type: Functional, Dynamic, Automatic
   Initial State: Trip in "in_progress" with all passengers dropped off.
   Input: Complete trip request from driver.
   Output: Trip marked "completed" with earnings summary; incomplete passengers block completion.
   Test Case Derivation: Verifies trip completion logic and earnings calculation.
   How test will be performed: Vitest in `apps/api/trpc/routers/__tests__/trip.test.ts`.

8. **test-ut-trip-8**
   Type: Functional, Dynamic, Automatic
   Initial State: Authenticated user with mocked database.
   Input: Valid address with latitude and longitude coordinates.
   Output: Default address saved/updated via upsert.
   Test Case Derivation: Verifies that users can persist their default pickup/dropoff

address.
How test will be performed: Vitest in `apps/api/trpc/routers/__tests__/location.test.ts`

9. **test-ut-trip-9**
   Type: Functional, Dynamic, Automatic
   Initial State: Authenticated user with mocked database.
   Input: Address string without latitude/longitude coordinates.
   Output: Validation error thrown; no database write occurs.
   Test Case Derivation: Ensures coordinate validation prevents incomplete location data from being saved.
   How test will be performed: Vitest in `apps/api/trpc/routers/__tests__/location.test.ts`

10. **test-ut-trip-10**
    Type: Functional, Dynamic, Automatic
    Initial State: Mocked Google Maps client.
    Input: Valid address string ("McMaster University").
    Output: Correct latitude/longitude coordinates returned.
    Test Case Derivation: Validates geocoding integration returns expected coordinate pairs for trip origin/destination.
    How test will be performed: Vitest in `apps/api/services/__tests__/googlemaps.test.ts`.

11. **test-ut-trip-11**
    Type: Functional, Dynamic, Automatic
    Initial State: Mocked Google Maps client.
    Input: Origin/destination coordinates with optional waypoints.
    Output: Correct total duration, distance, and waypoint ordering returned; empty routes throw error.
    Test Case Derivation: Verifies route calculation logic including multi-leg and optimized waypoint scenarios.
    How test will be performed: Vitest in `apps/api/services/__tests__/googlemaps.test.ts`.

12. **test-ut-trip-12**
    Type: Functional, Dynamic, Automatic
    Initial State: Mocked database with pending trip request.
    Input: Driver rejects a pending trip request.

Output: Request status updated to "rejected"; non-owner drivers are rejected.
Test Case Derivation: Validates the reject request flow and authorization checks.
How test will be performed: Vitest in `apps/api/trpc/routers/__tests__/trip.test.ts`.

13. **test-ut-trip-13**
    Type: Functional, Dynamic, Automatic
    Initial State: Mocked database with accepted trip request and payment hold.
    Input: Rider cancels their own accepted request.
    Output: Request cancelled; `bookedSeats` decremented; payment hold released.
    Test Case Derivation: Ensures cancellation correctly reverses seat allocation and payment authorization.
    How test will be performed: Vitest in `apps/api/trpc/routers/__tests__/trip.test.ts`.

14. **test-ut-trip-14**
    Type: Functional, Dynamic, Automatic
    Initial State: Mocked database with accepted request on an in-progress trip.
    Input: Rider confirms they are at the pickup location.
    Output: `riderPickupConfirmedAt` timestamp set; wrong rider or invalid status is rejected.
    Test Case Derivation: Validates the rider pickup confirmation flow.
    How test will be performed: Vitest in `apps/api/trpc/routers/__tests__/trip.test.ts`.

15. **test-ut-trip-15**
    Type: Functional, Dynamic, Automatic
    Initial State: Mocked database with rider who has no Stripe payment method on file.
    Input: Rider attempts to create a trip request.
    Output: Request rejected with a meaningful error directing the user to add a payment method.
    Test Case Derivation: Ensures riders cannot book rides without a payment method, protecting driver earnings.

How test will be performed: Vitest in `apps/api/trpc/routers/__tests__/trip.test.ts`.

16. **test-ut-trip-16**
    Type: Functional, Dynamic, Automatic
    Initial State: Mocked Stripe SDK configured to return a payment hold failure.
    Input: Driver accepts a trip request.
    Output: Request NOT accepted; error propagated; trip status remains unchanged.
    Test Case Derivation: Validates that a failed payment authorization prevents the acceptance from completing.
    How test will be performed: Vitest in `apps/api/trpc/routers/__tests__/trip.test.ts`.

17. **test-ut-trip-17**
    Type: Functional, Dynamic, Automatic
    Initial State: Trip with one already-accepted rider; mocked Stripe SDK.
    Input: Driver accepts a second rider.
    Output: Payment hold for first rider updated to reflect the multi-passenger discount tier.
    Test Case Derivation: Validates the retroactive discount logic applied when additional passengers join.
    How test will be performed: Vitest in `apps/api/trpc/routers/__tests__/trip.test.ts`.

18. **test-ut-trip-18**
    Type: Functional, Dynamic, Automatic
    Initial State: Active trip with accepted riders and active payment holds; mocked Stripe SDK.
    Input: Driver cancels the trip.
    Output: Trip cancelled; all accepted riders' payment holds released via `cancelPaymentHold`.
    Test Case Derivation: Ensures riders are not charged when a driver cancels an active trip.
    How test will be performed: Vitest in `apps/api/trpc/routers/__tests__/trip.test.ts`.

19. **test-ut-trip-19**

Type: Functional, Dynamic, Automatic
Initial State: Completed trip with multiple passengers.
Input: Driver completes the trip.
Output: Earnings summary contains correct `totalEarningsCents`, `passengerCount`, and per-passenger breakdown.
Test Case Derivation: Verifies the financial summary calculation on trip completion.
How test will be performed: Vitest in `apps/api/trpc/routers/__tests__/trip.test.ts`.

20. **test-ut-trip-20**
    Type: Functional, Dynamic, Automatic
    Initial State: Mocked database with requests for multiple trips and riders.
    Input: Driver queries requests for their specific trip; rider queries their own requests.
    Output: Driver sees only their trip's requests; rider sees only their own requests; cancelled entries filtered out.
    Test Case Derivation: Validates the filtered request retrieval logic for both driver and rider perspectives.
    How test will be performed: Vitest in `apps/api/trpc/routers/__tests__/trip.test.ts`.

21. **test-ut-trip-21**
    Type: Functional, Dynamic, Automatic
    Initial State: Completed trip with valid rider and driver Connect account; mocked Stripe SDK.
    Input: Rider submits a tip within valid bounds ($0.50–$500); attempt with out-of-bounds amount.
    Output: Valid tip processed and recorded; out-of-bounds amount rejected with validation error.
    Test Case Derivation: Validates tip submission input constraints and Stripe processing flow.
    How test will be performed: Vitest in `apps/api/trpc/routers/__tests__/trip.test.ts`.

22. **test-ut-trip-22**
    Type: Functional, Dynamic, Automatic
    Initial State: Mocked database with a completed trip owned by the

authenticated driver, and a rider who participated.
Input: Driver submits a rating (1–5) for a rider on the completed trip.
Output: Returns {`success: true`}.
Test Case Derivation: Validates the driver-rates-rider flow for completed trips.
How test will be performed: Vitest in `apps/api/trpc/routers/__tests__/trip.test.ts`.

23. **test-ut-trip-23**
    Type: Functional, Dynamic, Automatic
    Initial State: Mocked database with a completed trip owned by a different driver.
    Input: Non-owner driver attempts to rate a rider.
    Output: FORBIDDEN error thrown.
    Test Case Derivation: Ensures only the trip's driver can rate riders.
    How test will be performed: Vitest in `apps/api/trpc/routers/__tests__/trip.test.ts`.

24. **test-ut-trip-24**
    Type: Functional, Dynamic, Automatic
    Initial State: Mocked database with a non-completed (active) trip.
    Input: Driver attempts to rate a rider.
    Output: BAD_REQUEST error indicating trip must be completed.
    Test Case Derivation: Validates the trip completion guard on rider reviews.
    How test will be performed: Vitest in `apps/api/trpc/routers/__tests__/trip.test.ts`.

25. **test-ut-trip-25**
    Type: Functional, Dynamic, Automatic
    Initial State: Mocked database with a completed trip but rider was not a participant.
    Input: Driver attempts to rate a rider who was not on the trip.
    Output: NOT_FOUND error indicating rider request not found.
    Test Case Derivation: Ensures drivers can only rate riders who actually participated in the trip.
    How test will be performed: Vitest in `apps/api/trpc/routers/__tests__/trip.test.ts`.

26. **test-ut-trip-26**
    Type: Functional, Dynamic, Automatic
    Initial State: Mocked database with pending/active trips and date range.
    Input: Query for available trips with `startDate` filter.
    Output: Only trips departing on or after `startDate` are returned.
    Test Case Derivation: Validates the date range filtering on available trip queries.
    How test will be performed: Vitest in `apps/api/trpc/routers/__tests__/trip.test.ts`.

27. **test-ut-trip-27**
    Type: Functional, Dynamic, Automatic
    Initial State: Mocked database with pending/active trips and date range.
    Input: Query for available trips with `endDate` filter.
    Output: Only trips departing on or before `endDate` are returned.
    Test Case Derivation: Validates the upper bound date filtering on available trip queries.
    How test will be performed: Vitest in `apps/api/trpc/routers/__tests__/trip.test.ts`.

28. **test-ut-trip-28**
    Type: Functional, Dynamic, Automatic
    Initial State: Mocked database with a pending trip that has accepted riders.
    Input: Driver calls `fixTripStatus` for the stuck trip.
    Output: Trip status transitions from "pending" to "active"; `fixedCount` is 1.
    Test Case Derivation: Validates the administrative fix for trips stuck in pending status.
    How test will be performed: Vitest in `apps/api/trpc/routers/__tests__/trip.test.ts`.

29. **test-ut-trip-29**
    Type: Functional, Dynamic, Automatic
    Initial State: Mocked database with no stuck trips.
    Input: Driver calls `fixTripStatus`.
    Output: Returns {`fixedCount: 0`} with empty `fixedTrips` array.

Test Case Derivation: Validates the no-op case when no trips need fixing.
How test will be performed: Vitest in `apps/api/trpc/routers/__tests__/trip.test.ts`.

30. **test-ut-trip-30**
    Type: Functional, Dynamic, Automatic
    Initial State: Mocked database with an accepted trip request owned by the authenticated rider.
    Input: Rider calls `confirmRiderPickup` for their request.
    Output: `riderPickupConfirmedAt` timestamp set; updated request returned.
    Test Case Derivation: Validates the rider pickup confirmation success path.
    How test will be performed: Vitest in `apps/api/trpc/routers/__tests__/trip.test.ts`.

31. **test-ut-trip-31**
    Type: Functional, Dynamic, Automatic
    Initial State: Mocked database with an accepted trip request owned by a different rider.
    Input: Wrong rider attempts to confirm pickup.
    Output: FORBIDDEN error thrown.
    Test Case Derivation: Ensures only the request owner can confirm their own pickup.
    How test will be performed: Vitest in `apps/api/trpc/routers/__tests__/trip.test.ts`.

32. **test-ut-trip-32**
    Type: Functional, Dynamic, Automatic
    Initial State: Mocked database with a pending (not accepted) trip request.
    Input: Rider attempts to confirm pickup on a non-accepted request.
    Output: BAD_REQUEST error indicating request must be accepted first.
    Test Case Derivation: Validates the status guard on pickup confirmation.
    How test will be performed: Vitest in `apps/api/trpc/routers/__tests__/trip.test.ts`.

33. **test-ut-trip-33**
    Type: Functional, Dynamic, Automatic
    Initial State: Mocked database with an active trip and accepted riders with push tokens.
    Input: Driver starts the trip via `startTrip`.
    Output: Trip status transitions to "in_progress"; `sendTripNotification` called for accepted riders.
    Test Case Derivation: Validates that starting a trip triggers push notifications to all accepted riders.
    How test will be performed: Vitest in `apps/api/trpc/routers/__tests__/trip.test.ts`.

34. **test-ut-trip-34**
    Type: Functional, Dynamic, Automatic
    Initial State: Mocked database with an accepted trip request and associated trip with booked seats; mocked payment service.
    Input: Rider cancels their accepted request via `cancelTripRequest`.
    Output: Request cancelled; `bookedSeats` decremented; `cancelPaymentHold` called.
    Test Case Derivation: Validates that cancelling an accepted request correctly releases the seat and payment hold.
    How test will be performed: Vitest in `apps/api/trpc/routers/__tests__/trip.test.ts`.

4.2.4 Matching Module

Test file: `apps/api/services/__tests__/matchmaking_service.test.ts`

1. **test-ut-match-1**
    Type: Functional, Dynamic, Automatic
    Initial State: Mock database with overlapping driver and rider schedules and routes.
    Input: Match request from a verified rider.
    Output: List of compatible drivers returned, properly filtering active pending requests.
    Test Case Derivation: Confirms the core matching algorithm pairs users based on constraints (FR-3).
    How test will be performed: Vitest in `apps/api/services/__tests__/matchmaking_service.`

2. **test-ut-match-2**
   Type: Functional, Dynamic, Automatic
   Initial State: Mock database containing only drivers with disjoint schedules/routes.
   Input: Match request from a verified rider.
   Output: Empty list response.
   Test Case Derivation: Ensures false positive matches are not generated when constraints are unmet.
   How test will be performed: Vitest in `apps/api/services/__tests__/matchmaking_service.`

3. **test-ut-match-3**
   Type: Functional, Dynamic, Automatic
   Initial State: Mocked database with real drivers who have existing reviews.
   Input: Rider queries for matches via `findMatches` router procedure.
   Output: Matches returned with real driver ratings looked up from the reviews table.
   Test Case Derivation: Validates that the matchmaking router enriches match results with actual driver ratings.
   How test will be performed: Vitest in `apps/api/trpc/routers/__tests__/matchmaking.tes`

4. **test-ut-match-4**
   Type: Functional, Dynamic, Automatic
   Initial State: Mocked matchmaking service returning dummy driver matches.
   Input: Rider queries for matches with `includeDummyMatches` enabled.
   Output: Dummy matches returned unmodified (no rating lookup for `dummy-*` driver IDs).
   Test Case Derivation: Ensures dummy matches bypass the review rating enrichment step.
   How test will be performed: Vitest in `apps/api/trpc/routers/__tests__/matchmaking.tes`

5. **test-ut-match-5**
   Type: Functional, Dynamic, Automatic
   Initial State: Mocked matchmaking service returning no matches.
   Input: Rider queries for matches.

Output: Empty array returned.
Test Case Derivation: Validates the empty result path in the matchmaking router.
How test will be performed: Vitest in `apps/api/trpc/routers/__tests__/matchmaking.tes`

6. **test-ut-match-6**
   Type: Functional, Dynamic, Automatic
   Initial State: Mocked database with a pending trip request owned by the authenticated rider.
   Input: Rider cancels their own request via `cancelRequest`.
   Output: Request status updated to "cancelled"; returns {`success: true`}.
   Test Case Derivation: Validates the basic cancellation flow for pending requests.
   How test will be performed: Vitest in `apps/api/trpc/routers/__tests__/matchmaking.tes`

7. **test-ut-match-7**
   Type: Functional, Dynamic, Automatic
   Initial State: Mocked database with an accepted trip request and a trip with booked seats.
   Input: Rider cancels their accepted request.
   Output: Request cancelled; `bookedSeats` decremented by 1 on the associated trip.
   Test Case Derivation: Validates that accepted request cancellation correctly frees the booked seat.
   How test will be performed: Vitest in `apps/api/trpc/routers/__tests__/matchmaking.tes`

8. **test-ut-match-8**
   Type: Functional, Dynamic, Automatic
   Initial State: Mocked database with a trip request owned by a different rider.
   Input: Rider attempts to cancel another rider's request.
   Output: FORBIDDEN error thrown.
   Test Case Derivation: Ensures riders can only cancel their own requests.
   How test will be performed: Vitest in `apps/api/trpc/routers/__tests__/matchmaking.tes`

4.2.5 Notification Module

Test file: `apps/api/services/__tests__/notification.test.ts`

1. **test-ut-notif-1**
   Type: Functional, Dynamic, Automatic
   Initial State: Mocked Expo push notification SDK.
   Input: List of recipients (some with valid tokens, some without).
   Output: Only valid Expo push tokens receive notifications; invalid tokens are silently filtered.
   Test Case Derivation: Ensures notification delivery is resilient to invalid push tokens.
   How test will be performed: Vitest in `apps/api/services/__tests__/notification.test.ts`

4.2.6 Rating & Feedback Module

Test file: `apps/api/trpc/routers/__tests__/reviews.test.ts`

1. **test-ut-review-1**
   Type: Functional, Dynamic, Automatic
   Initial State: Mocked database with completed trip.
   Input: Valid rating (1–5) for another user on a completed trip.
   Output: Review persisted to database; self-rating and duplicate ratings are rejected.
   Test Case Derivation: Validates the post-trip rating flow and business rule constraints.
   How test will be performed: Vitest in `apps/api/trpc/routers/__tests__/reviews.test.ts`

2. **test-ut-review-2**
   Type: Functional, Dynamic, Automatic
   Initial State: Mocked database with existing reviews for a user.
   Input: Query for user's average score.
   Output: Correct average and review count returned; new user returns "New".
   Test Case Derivation: Verifies aggregate score calculation logic.
   How test will be performed: Vitest in `apps/api/trpc/routers/__tests__/reviews.test.ts`

4.2.7 Safety & Reporting Module

Test file: `apps/api/trpc/routers/__tests__/complaints.test.ts`

1. **test-ut-complaint-1**
   Type: Functional, Dynamic, Automatic
   Initial State: Authenticated user with mocked database.
   Input: Complaint content and target user ID.
   Output: Complaint persisted to the `complaints` table.
   Test Case Derivation: Validates the user reporting flow.
   How test will be performed: Vitest in `apps/api/trpc/routers/__tests__/complaints.test`

4.2.8 Payment & Cost Estimation Module

Test file: `apps/api/services/__tests__/payment.test.ts`

1. **test-ut-payment-1**
   Type: Functional, Dynamic, Automatic
   Initial State: Mocked Stripe SDK and database.
   Input: Valid Stripe customer creation parameters.
   Output: Stripe customer created and persisted to `stripeCustomers` table; existing customer returned on duplicate call.
   Test Case Derivation: Validates the idempotent customer creation flow.
   How test will be performed: Vitest in `apps/api/services/__tests__/payment.test.ts`.

2. **test-ut-payment-2**
   Type: Functional, Dynamic, Automatic
   Initial State: Mocked Stripe SDK.
   Input: Trip distance, duration, passenger count, and detour values.
   Output: Correct `totalCents`, `platformFeeCents` (15%), and `driverAmountCents` split.
   Test Case Derivation: Validates fare calculation arithmetic and platform fee deduction.
   How test will be performed: Vitest in `apps/api/services/__tests__/payment.test.ts`.

3. **test-ut-payment-3**
   Type: Functional, Dynamic, Automatic

Initial State: Mocked Stripe SDK with valid customer and Connect account.
Input: Payment hold creation for a trip request.
Output: Stripe PaymentIntent created with correct amount, application fee, and transfer destination.
Test Case Derivation: Validates the Stripe Connect payment authorization flow.
How test will be performed: Vitest in `apps/api/services/__tests__/payment.test.ts`.

4. **test-ut-payment-4**
   Type: Functional, Dynamic, Automatic
   Initial State: Mocked Stripe SDK with existing PaymentIntent.
   Input: Capture payment for a completed trip request.
   Output: PaymentIntent captured and database status updated to "captured".
   Test Case Derivation: Validates the payment capture flow triggered on passenger dropoff.
   How test will be performed: Vitest in `apps/api/services/__tests__/payment.test.ts`.

5. **test-ut-payment-5**
   Type: Functional, Dynamic, Automatic
   Initial State: Mocked Stripe SDK with existing PaymentIntent.
   Input: Cancel payment hold for a cancelled trip request.
   Output: PaymentIntent cancelled and database status updated to "cancelled".
   Test Case Derivation: Ensures payment holds are properly released on trip or request cancellation.
   How test will be performed: Vitest in `apps/api/services/__tests__/payment.test.ts`.

6. **test-ut-payment-6**
   Type: Functional, Dynamic, Automatic
   Initial State: Mocked Stripe SDK with completed trip.
   Input: Rider submits a tip with valid amount.
   Output: Tip processed via Stripe and recorded in the `tips` table.
   Test Case Derivation: Validates the tip processing flow and database persistence.

How test will be performed: Vitest in `apps/api/services/__tests__/payment.test.ts`.

7. **test-ut-payment-7**
   Type: Functional, Dynamic, Automatic
   Initial State: Mocked database and Stripe SDK.
   Input: Authenticated user calls `createSetupIntent`.
   Output: Stripe customer created/retrieved; setup intent client secret returned.
   Test Case Derivation: Validates the setup intent creation flow at the router level.
   How test will be performed: Vitest in `apps/api/trpc/routers/__tests__/payment-router.t`

8. **test-ut-payment-8**
   Type: Functional, Dynamic, Automatic
   Initial State: Mocked database returning no user.
   Input: Non-existent user calls `createSetupIntent`.
   Output: NOT_FOUND error thrown.
   Test Case Derivation: Validates error handling when user lookup fails.
   How test will be performed: Vitest in `apps/api/trpc/routers/__tests__/payment-router.t`

9. **test-ut-payment-9**
   Type: Functional, Dynamic, Automatic
   Initial State: Mocked Stripe SDK with existing customer and payment methods.
   Input: Authenticated user queries `getPaymentMethods`.
   Output: List of methods with `brand`, `last4`, `isDefault` correctly mapped; default method sorted first.
   Test Case Derivation: Validates payment method retrieval and default flag logic.
   How test will be performed: Vitest in `apps/api/trpc/routers/__tests__/payment-router.t`

10. **test-ut-payment-10**
    Type: Functional, Dynamic, Automatic
    Initial State: Mocked Stripe SDK returning no customer ID.
    Input: User with no Stripe customer queries `getPaymentMethods`.

Output: Empty methods array returned; `hasPaymentMethod` is `false`.
Test Case Derivation: Validates the empty state for new users.
How test will be performed: Vitest in `apps/api/trpc/routers/__tests__/payment-router.t`

11. **test-ut-payment-11**
    Type: Functional, Dynamic, Automatic
    Initial State: Mocked Stripe SDK with valid payment method.
    Input: User calls `deletePaymentMethod` with a valid ID.
    Output: Payment method detached; returns {`success: true`}.
    Test Case Derivation: Validates the payment method deletion flow.
    How test will be performed: Vitest in `apps/api/trpc/routers/__tests__/payment-router.t`

12. **test-ut-payment-12**
    Type: Functional, Dynamic, Automatic
    Initial State: Mocked Stripe SDK with no matching method.
    Input: User calls `deletePaymentMethod` with a non-existent ID.
    Output: NOT_FOUND error thrown.
    Test Case Derivation: Validates error handling for invalid payment method deletion.
    How test will be performed: Vitest in `apps/api/trpc/routers/__tests__/payment-router.t`

13. **test-ut-payment-13**
    Type: Functional, Dynamic, Automatic
    Initial State: Mocked payment service.
    Input: User calls `setDefaultPaymentMethod` and `hasPaymentMethod`.
    Output: Default set successfully; `hasPaymentMethod` returns correct boolean.
    Test Case Derivation: Validates simple pass-through router procedures.
    How test will be performed: Vitest in `apps/api/trpc/routers/__tests__/payment-router.t`

14. **test-ut-payment-14**
    Type: Functional, Dynamic, Automatic
    Initial State: Mocked database and Stripe SDK.
    Input: Driver calls `createConnectOnboarding` with valid URLs.
    Output: Connect account created; onboarding URL returned.

Test Case Derivation: Validates the Stripe Connect onboarding initiation flow.
How test will be performed: Vitest in `apps/api/trpc/routers/__tests__/payment-router.t`

15. **test-ut-payment-15**
    Type: Functional, Dynamic, Automatic
    Initial State: Mocked payment service.
    Input: Driver queries `getConnectStatus`.
    Output: Correct Connect account status returned.
    Test Case Derivation: Validates the Connect status retrieval pass-through.
    How test will be performed: Vitest in `apps/api/trpc/routers/__tests__/payment-router.t`

16. **test-ut-payment-16**
    Type: Functional, Dynamic, Automatic
    Initial State: Mocked database with completed trip and valid rider request.
    Input: Rider calls `submitTip` with a valid amount.
    Output: Tip processed successfully; returns {`success: true`}.
    Test Case Derivation: Validates the tip submission flow at the router level.
    How test will be performed: Vitest in `apps/api/trpc/routers/__tests__/payment-router.t`

17. **test-ut-payment-17**
    Type: Functional, Dynamic, Automatic
    Initial State: Mocked database with non-completed trip.
    Input: Rider calls `submitTip` for an active trip.
    Output: BAD_REQUEST error indicating tips are only for completed trips.
    Test Case Derivation: Validates the trip completion guard on tip submission.
    How test will be performed: Vitest in `apps/api/trpc/routers/__tests__/payment-router.t`

18. **test-ut-payment-18**
    Type: Functional, Dynamic, Automatic

Initial State: Mocked database with driver's completed trips, payments, and riders.
Input: Driver queries `getDriverPayoutHistory`.
Output: Payment list with rider names, amounts, and summary containing `totalEarningsCents`, `totalPlatformFeeCents`, and `pendingCents`.
Test Case Derivation: Validates the driver payout history query and earnings summary calculation.
How test will be performed: Vitest in `apps/api/trpc/routers/__tests__/payment-router.t`

19. **test-ut-payment-19**
    Type: Functional, Dynamic, Automatic
    Initial State: Mocked database with rider's trip payments and tips.
    Input: Rider queries `getRiderPaymentHistory`.
    Output: Payment list with driver names, tip history, and summary containing `totalSpentCents`, `totalTipsCents`, and `rideCount`.
    Test Case Derivation: Validates the rider payment history query and spending summary calculation.
    How test will be performed: Vitest in `apps/api/trpc/routers/__tests__/payment-router.t`

20. **test-ut-payment-20**
    Type: Functional, Dynamic, Automatic
    Initial State: Mocked payment service.
    Input: Query `getPaymentStatus` with a valid trip request ID.
    Output: Correct payment status returned from the service layer.
    Test Case Derivation: Validates the payment status retrieval pass-through.
    How test will be performed: Vitest in `apps/api/trpc/routers/__tests__/payment-router.t`

4.2.9 Pricing Module

Test file: `apps/api/services/__tests__/pricing_service.test.ts`

1. **test-ut-pricing-1**
   Type: Functional, Dynamic, Automatic
   Initial State: System parameters ready.
   Input: Valid trip distance, active rate constants, and detour offsets.

Output: Correctly computed shared cost value, with detour surcharge capped at 25%.
Test Case Derivation: Validates the arithmetic consistency of the cost-sharing logic (FR-4).
How test will be performed: Vitest in `apps/api/services/__tests__/pricing_service.test`

2. **test-ut-pricing-2**
   Type: Functional, Dynamic, Automatic
   Initial State: System parameters ready.
   Input: Trip capacity variables involving 1, 2, or 3+ passengers.
   Output: Correct exponential discounts are applied to base rates.
   Test Case Derivation: Confirms proper passenger capacity discounts for recurring rides.
   How test will be performed: Vitest in `apps/api/services/__tests__/pricing_service.test`

4.2.10 Database Module

Test file: `packages/db/tests/schema.test.ts`

1. **test-ut-schema-1**
   Type: Structural, Dynamic, Automatic
   Initial State: Test database connection pool.
   Input: Insert a new user with minimal fields (id, name, email).
   Output: User created with correct defaults (`emailVerified: false`, auto-generated `createdAt`).
   Test Case Derivation: Verifies that schema defaults are correctly applied by the database.
   How test will be performed: Vitest in `packages/db/tests/schema.test.ts`.

2. **test-ut-schema-2**
   Type: Structural, Dynamic, Automatic
   Initial State: Test database with no matching user.
   Input: Insert a profile with a non-existent `userId`.
   Output: Foreign key constraint violation error thrown.
   Test Case Derivation: Ensures referential integrity is enforced at the database level.

How test will be performed: Vitest in `packages/db/tests/schema.test.ts`.

4.3 Tests for Nonfunctional Requirements

Unit tests are not the primary method for verifying NFRs (see Section 3).

4.4 Traceability Between Test Cases and Modules

Table 2 maps each unit test to its corresponding MIS module and implementation status.

Table 2: Unit Test Traceability Matrix

Test ID	**MIS Module**	**Test File**
test-ut-auth-1, 2	Auth & Verification	validators.test.ts
test-ut-auth-3	Auth & Verification	client.test.ts
test-ut-profile-1, 2	User Profile	profile.test.ts
test-ut-profile-3, 4	User Profile	utils/index.test.ts
test-ut-profile-5–10	User Profile	profile.test.ts
test-ut-trip-1–11	Route & Trip	trip/location/googlemaps.test.ts
test-ut-trip-12–21	Route & Trip	trip.test.ts
test-ut-trip-22–34	Route & Trip	trip.test.ts
test-ut-match-1, 2	Matching	matchmaking_service.test.ts
test-ut-match-3–8	Matching	matchmaking.test.ts
test-ut-notif-1	Notification	notification.test.ts
test-ut-review-1, 2	Rating & Feedback	reviews.test.ts
test-ut-complaint-1	Safety & Reporting	complaints.test.ts
test-ut-payment-1–6	Payment & Cost Est.	payment.test.ts
test-ut-payment-7–20	Payment & Cost Est.	payment-router.test.ts
test-ut-pricing-1, 2	Pricing	pricing_service.test.ts
test-ut-schema-1, 2	Database	schema.test.ts

References

Burhanuddin Kharodawala, Sarim Zia, Swesan Pathmanathan, Aidan Froggatt, and Hamzah Rawasia. Development plan. https://github.com/...,

2025a.

Burhanuddin Kharodawala, Sarim Zia, Swesan Pathmanathan, Aidan Froggatt, and Hamzah Rawasia. Hazard analysis. https://github.com/..., 2025b.

Burhanuddin Kharodawala, Sarim Zia, Swesan Pathmanathan, Aidan Froggatt, and Hamzah Rawasia. Module guide. https://github.com/..., 2025c.

Burhanuddin Kharodawala, Sarim Zia, Swesan Pathmanathan, Aidan Froggatt, and Hamzah Rawasia. Module interface specification. https://github.com/..., 2025d.

Burhanuddin Kharodawala, Sarim Zia, Swesan Pathmanathan, Aidan Froggatt, and Hamzah Rawasia. Problem statement and goals. https://github.com/..., 2025e.

Burhanuddin Kharodawala, Sarim Zia, Swesan Pathmanathan, Aidan Froggatt, and Hamzah Rawasia. System requirements specification. https://github.com/..., 2025f.

Burhanuddin Kharodawala, Sarim Zia, Swesan Pathmanathan, Aidan Froggatt, and Hamzah Rawasia. Usability testing report. https://github.com/..., 2025g.

5 Appendix

5.1 Usability Testing Plan

Usability testing will be conducted at the end of the first stage of development of the application. This will be used to assess and judge the user's compatibility with the application. This section will include a plan for conducting this test.
The goal is to conduct usability testing right after the development of the MVP of Hitchly. The MVP (minimal viable product) includes the ride matching algorithm, and user authentication. The plan is to find a group of students and staff to navigate through the application on their own and ask them for any feedback they may have. Secondly, ask the users a set of questions and observe their expressions; the time it takes them to complete a given task and features that may confuse them. Below is a list of possible questions/tasks a user must complete to test the user compatibility of the application:

- Launch the application and register on the application.
- Navigate to the user profile and add your timetable to the application. Update some information on your profile.
- Navigate to the list of matched drivers/riders in the application.
- Select your profile type, Drivers or Riders.
- For Riders, click on the matched driver and access their user information.
- For Drivers, click on the matched rider and access their user information.
- Log out of the application and re-login to the application with the registered credentials in a given time constraint.

Here are the main objectives of the test:

- Users find the process of creating an account intuitive and easy.
- Users are able to verify their emails quickly.

- Users are able to navigate through the various sections of the app without any problem.
- Users address any discomfort they have while using the application.
- Users address any errors they face while using the application.

This brief test plan provides a structured approach for analyzing user's interaction with the application and understanding the key areas of improvement. Users will be monitored using a time metric to check if a user is able to complete a task within a given time constraint, ensuring efficiency. Moreover, users will be asked for feedback after each task they perform to get as much feedback as possible to improve the user experience.

Appendix — Reflection

The information in this section will be used to evaluate the team members on the graduate attribute of Lifelong Learning.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. **What went well while writing this deliverable?**
   (Swesan) The structure of the V&V plan came together smoothly once I reviewed the example templates and understood what was expected. I was able to clearly link each test to the functional and non-functional requirements from Section 1.4 (G.4), which made the writing process more organized. Formatting, tone consistency, and version control all went well, and I'm happy with how cohesive the final document feels.

   (Burhanuddin) This document was very helpful in setting the basis for identifying and documenting the testing approaches for our capstone project. Specifically, the list of documentation was a very critical section that allowed me to understand the importance of each document and their connection to the specific tests. Overall, we had an in-depth discussion of the testing approaches we wanted to consider and documented them with full clarity.

   (Hamzah) A few things that went well were that this document tied back to previous documents we already completed, so being able to see everything come together under the project was nice. I worked on the

Plan section, so having a clear project scope and defined features made it easier to brainstorm what needed to be verified and validated for each part of the project. It was also nice to have some reference/example documents to look at to get a better idea of the expectations in some sections.

(Sarim) Working on one section with another team member meant we had to ensure we were fairly dividing the section. It also meant that some parts would be dependent on the other team members to finish before they could be started. We were able to deal with this well by discussing our schedules early on and setting key deadlines so neither of us suffered from delays by the other.

2. **What pain points did you experience during this deliverable, and how did you resolve them?**
(Swesan) The main difficulty was determining how detailed each test case should be and distinguishing between verification and validation tasks. At first, I wrote overly detailed descriptions, but after reviewing previous examples and clarifying expectations from the course materials, I refined the structure to focus on clarity and traceability. Once I aligned the test coverage with the proof-of-concept scope of Hitchly, the process became much more straightforward.

(Burhanuddin) Initially, the documentation had a few areas/sections that were very ambiguous in what they wanted. They lacked clarity in the description of the sections. Some of the descriptions were also outdated, which caused some confusion. Example, section 2 had a subsection for challenges which is part of an old rubric. Through the TA meetings, we were able to clarify what the description meant.

(Hamzah) At first it was a bit hard to understand what was expected in the Plan sections, in terms of what can be actionable, and there seemed to be a lot of overlap between sections. Also, since our team does not have a supervisor for the project, it felt like we were missing out on a big aspect of the verification process. After speaking with the TA, I was able to get some more clarity on what parts can overlap

between sections, and also how we can incorporate stakeholders into the plan instead of a supervisor.

(Sarim) Our team had numerous questions planned for the TA meeting, however, due to traffic delays, most of us were unable to reach on time. We were able to communicate this early to the team members who were present and get our deliverable questions across even with the inability to attend on time. For me, these questions helped clear up ambiguity in the Plan section and ensured I was able to have a clear understanding while writing the section.

3. **What knowledge and skills will you need to acquire to successfully complete the verification and validation of your project?**
(Swesan) I need to deepen my understanding of dynamic testing and automation. Specifically, I want to improve at writing reproducible automated test scripts for functional verification and gain familiarity with load testing tools to assess performance under peak usage conditions. These skills will help ensure Hitchly's matching and verification features are both reliable and scalable.

(Burhanuddin) As I am fairly new to app development, I need to learn all types of web development testing frameworks. I also need to understand them in Typescript, a language we will be using for the development of the application.

(Hamzah) An important skill the team will need to learn is Jest, which will be the primary unit testing framework we use to test our codebase. Writing unit tests with good coverage and effectiveness is crucial for the project, so it's important to be familiar with the tools that will be used.

(Sarim) I think when it comes to verification, knowing the tool is important, however it is also important to know theory about what type of tests we can and should perform alongside any additional theory that could help provide complete verification coverage. However, the last time our team members took a class related to verification testing

was years ago, so it may be an aspect we need a refresher on.

(Swesan) The structure of the V&V plan came together smoothly once I reviewed the example templates and understood what was expected. I was able to clearly link each test to the functional and non-functional requirements from Section 1.4 (G.4), which made the writing process more organized. Formatting, tone consistency, and version control all went well, and I'm happy with how cohesive the final document feels.

(Aidan) I found that structuring the V&V plan around clear verification tiers (SRS, design, implementation, and validation) made the writing process straightforward. Aligning test cases directly with functional and non-functional requirements improved traceability and reduced redundancy. Integrating automation tools like Jest and GitHub Actions into the plan also felt natural since these align closely with our existing workflow. Overall, this deliverable came together efficiently once the structure and testing philosophy were defined.

4. **What pain points did you experience during this deliverable, and how did you resolve them?**
   (Swesan) The main difficulty was determining how detailed each test case should be and distinguishing between verification and validation tasks. At first, I wrote overly detailed descriptions, but after reviewing previous examples and clarifying expectations from the course materials, I refined the structure to focus on clarity and traceability. Once I aligned the test coverage with the proof-of-concept scope of Hitchly, the process became much more straightforward.
   (Aidan) One challenge was balancing feasibility with completeness — the temptation was to include every possible validation step, but time and scope required focusing on what could realistically be tested this term. I also found it tricky to document test coverage without finalizing the detailed design. To resolve this, I focused on defining testing strategies conceptually and left module-specific details as placeholders for post-design updates.

5. **What knowledge and skills will you need to acquire to successfully complete the verification and validation of your project?**
   (Swesan) I need to deepen my understanding of dynamic testing and

automation. Specifically, I want to improve at writing reproducible automated test scripts for functional verification and gain familiarity with load testing tools to assess performance under peak usage conditions. These skills will help ensure Hitchly's matching and verification features are both reliable and scalable.
(Aidan) I want to strengthen my understanding of end-to-end testing workflows and CI/CD validation. Specifically, I need to improve at automating test pipelines that integrate unit, integration, and mutation testing with coverage reporting. I also plan to deepen my knowledge of usability testing methodologies and how to quantify user satisfaction during beta testing.

6. **For each of the knowledge areas and skills identified in the previous question, what are at least two approaches to acquiring the knowledge or mastering the skill? Which will you pursue, and why?**
(Swesan) To build my dynamic testing and automation skills, I can:
(1) Review documentation and tutorials for testing frameworks and testing utilities.
(2) Develop small-scale automated test cases directly within the Hitchly repository to gain hands-on experience.

I will prioritize the second approach since applying testing concepts directly in code helps uncover integration issues early and strengthens confidence in both the testing process and implementation reliability.
(Burhanuddin) I will first look at language documentation and understand the basics of technology. Secondly, I will go through some video tutorials and get some hands-on practice with the desired testing tools.
(Hamzah) One approach is to learn the important principles and strategies of good unit tests and writing test suites for a project, as having the fundamental understanding is the most important aspect. Once we are comfortable with that, we can follow video tutorials and look at examples of writing unit tests using Jest to get familiar with the framework.
(Sarim) The best approach would be to review any relevant course material from past years. This is a good approach because it would simply be a quick refresher in a format, we are familiar with, rather than learning something with a new format/approach. However, another option

is to follow any online sources such as tech blogs that show how to provide complete system coverage.

(Aidan) For CI/CD and automated test coverage, I can: (1) Experiment with GitHub Actions to create incremental test pipelines that validate each pull request automatically. (2) Study testing patterns from large-scale open-source monorepos to understand practical approaches to test orchestration. For usability validation, I can: (1) Research UX testing frameworks and heuristics to design structured surveys and A/B test flows. (2) Conduct real-world pilot tests with early users to collect actionable feedback. I plan to prioritize hands-on experimentation with automated pipelines since this will directly improve code reliability and streamline team workflows.